

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING																		
Program Name: B. Tech		Assignment Type: Lab	Academic Year: 2025-2026																	
Course Coordinator Name		Dr. Rishabh Mittal																		
Instructor(s) Name		<table border="1"> <tr><td>Mr. S Naresh Kumar</td></tr> <tr><td>Ms. B. Swathi</td></tr> <tr><td>Dr. Sasanko Shekhar Gantayat</td></tr> <tr><td>Mr. Md Sallauddin</td></tr> <tr><td>Dr. Mathivanan</td></tr> <tr><td>Mr. Y Srikanth</td></tr> <tr><td>Ms. N Shilpa</td></tr> <tr><td>Dr. Rishabh Mittal (Coordinator)</td></tr> <tr><td>Dr. R. Prashant Kumar</td></tr> <tr><td>Mr. Ankushavali MD</td></tr> <tr><td>Mr. B Viswanath</td></tr> <tr><td>Ms. Sujitha Reddy</td></tr> <tr><td>Ms. A. Anitha</td></tr> <tr><td>Ms. M.Madhuri</td></tr> <tr><td>Ms. Katherashala Swetha</td></tr> <tr><td>Ms. Velpula sumalatha</td></tr> <tr><td>Mr. Bingi Raju</td></tr> </table>		Mr. S Naresh Kumar	Ms. B. Swathi	Dr. Sasanko Shekhar Gantayat	Mr. Md Sallauddin	Dr. Mathivanan	Mr. Y Srikanth	Ms. N Shilpa	Dr. Rishabh Mittal (Coordinator)	Dr. R. Prashant Kumar	Mr. Ankushavali MD	Mr. B Viswanath	Ms. Sujitha Reddy	Ms. A. Anitha	Ms. M.Madhuri	Ms. Katherashala Swetha	Ms. Velpula sumalatha	Mr. Bingi Raju
Mr. S Naresh Kumar																				
Ms. B. Swathi																				
Dr. Sasanko Shekhar Gantayat																				
Mr. Md Sallauddin																				
Dr. Mathivanan																				
Mr. Y Srikanth																				
Ms. N Shilpa																				
Dr. Rishabh Mittal (Coordinator)																				
Dr. R. Prashant Kumar																				
Mr. Ankushavali MD																				
Mr. B Viswanath																				
Ms. Sujitha Reddy																				
Ms. A. Anitha																				
Ms. M.Madhuri																				
Ms. Katherashala Swetha																				
Ms. Velpula sumalatha																				
Mr. Bingi Raju																				
Course Code	23CS002PC304	Course Title	AI Assisted Coding																	
Year/Sem	III/II	Regulation	R23																	
Date and Day of Assignment	Week3 – Wednesday	Time(s)	23CSBTB01 To 23CSBTB52																	
Duration	2 Hours	Applicable to Batches	All batches																	
AssignmentNumber: 6.3(Present assignment number)/24(Total number of assignments)																				
Q.No.	Question	Expected Time to complete																		
1	Lab 6: AI-Based Code Completion – Classes, Loops, and Conditionals Lab Objectives • To explore AI-powered auto-completion features for core Python constructs such as classes, loops, and conditional statements. • To analyze how AI tools suggest logic for object-oriented programming and control structures.	Week3 - Wednesday																		

- To evaluate the correctness, readability, and completeness of AI-generated Python code.

Lab Outcomes (LOs)

After completing this lab, students will be able to:

- Use AI tools to generate and complete Python class definitions and methods.
- Understand and assess AI-suggested loop constructs for iterative tasks.
- Generate and evaluate conditional statements using AI-driven prompts.
- Critically analyze AI-assisted code for correctness, clarity, and efficiency.

Task Description #1: Classes (Student Class)

Scenario

You are developing a simple student information management module.

Prompt: Write a Python program that defines a class named Student with attributes name, roll number, and branch. Use a constructor to initialize these values. Add a method display_details() to print student information. In the main program, take name, roll number, and branch as user input, create a Student object, and display the details.

```

1 class Student:
2     print(f"Branch: {self.branch}")
3
4 if __name__ == "__main__":
5     # Read student details from user input
6     name = input("Enter student name: ").strip()
7     try:
8         roll_number = int(input("Enter roll number: ").strip())
9     except ValueError:
10        print("Invalid roll number entered. Defaulting to 0.")
11        roll_number = 0
12    branch = input("Enter branch: ").strip()
13
14    student = Student(name, roll_number, branch)
15    student.display_details()

```

Terminal Output:

```

PS C:\Users\sai\OneDrive\Desktop\AI> python student.py
Roll Number: 181
Branch: Computer Science
Enter student name: sai
Enter roll number: 2303A
Invalid roll number entered. Defaulting to 0.
Roll Number: 0
Name: sai
Branch: 2303A

```

Task Description #2: Loops (Multiples of a Number)

Scenario

You are writing a utility function to display multiples of a given number.

Prompt:

1 – For Loop

Write a Python function that asks the user to enter a number and prints the first 10 multiples of that number using a for loop.

2-While Loop

Rewrite the same program that prints the first 10 multiples of a number, but use a while loop instead of a for loop. Keep user input dynamic.

3- Analysis

Explain the difference between the for loop and while loop implementations for printing multiples of a number.

```
File Edit Selection View Go Run ... Q AI
EXPLORER
  multiples.py
  review_sentiment_few_shot.py
  collect_user_data.py
  sentiment_bias_mitigation.py
  product_recommender_transparency_fair...
  secure_logging.py
  simple_ml_model.py
  hpcc.py
  student.py
  multiples.py
  factorial_recursive.py
  hpcc.py
  hpcc.py
  Lab Assignment1.2.py
  lab1
  leap_year.py
  multiples.py
  product_recommender_transparency_fairness...
  review_sentiment_few_shot.py
  review_sentiment_one_shot.py
  review_sentiment_zero_shot.py
  review_sentiment.py
  example1st
  OUTLINE
  TIMELINE
  ZIP EXPLORER

MULTIPLES.PY
1 def print_first_10_multiples() -> None:
2     """Ask the user for a number and print its first 10 multiples."""
3     try:
4         n = int(input("Enter a number: ").strip())
5     except ValueError:
6         print("Invalid number entered.")
7         return
8     for i in range(1, 11):
9         print(n * i)
10
11
12
13 if __name__ == "__main__":
14     print_first_10_multiples()
15
PROBLEMS OUTPUT TERMINAL
PS C:\Users\saiiku\OneDrive\Desktop\AI> python multiples.py
Enter a number: 5
5
10
15
20
25
30
35
40
45
50
Ln 12, Col 21 Spaces: 4 UTF-8 CRLF Python conda (3.14.2) B4 Go Live 09:35 04-02-2026
```

```
File Edit Selection View Go Run ... Q AI
EXPLORER
  multiples_while.py
  collect_user_data.py
  sentiment_bias_mitigation.py
  product_recommender_transparency_fair...
  secure_logging.py
  simple_ml_model.py
  hpcc.py
  student.py
  multiples_while.py
  Assignment3.3.py
  factorial_recursive.py
  hpcc.py
  hpcc.py
  Lab Assignment1.2.py
  lab1
  leap_year.py
  multiples.py
  product_recommender_transparency_fairness...
  review_sentiment_few_shot.py
  review_sentiment_one_shot.py
  review_sentiment_zero_shot.py
  review_sentiment.py
  example1st
  OUTLINE
  TIMELINE
  ZIP EXPLORER

MULTIPLES_WHILE.PY
1 def print_first_10_multiples_while() -> None:
2     """Ask the user for a number and print its first 10 multiples using a while loop."""
3     try:
4         n = int(input("Enter a number: ").strip())
5     except ValueError:
6         print("Invalid number entered.")
7         return
8     i = 1
9     while i <= 10:
10        print(n * i)
11        i += 1
12
13
14 if __name__ == "__main__":
15     print_first_10_multiples_while()
16
PROBLEMS OUTPUT TERMINAL
PS C:\Users\saiiku\OneDrive\Desktop\AI> python multiples_while.py
Enter a number: 5
5
10
15
20
25
30
35
40
45
50
Ln 12, Col 15 Spaces: 4 UTF-8 CRLF Python conda (3.14.2) B4 Go Live 09:38 04-02-2026
```

```
File Edit Selection View Go Run ... Q AI
EXPLORER
  multiples_while.py
  multiples.py
  collect_user_data.py
  sentiment_bias_mitigation.py
  product_recommender_transparency_fair...
  secure_logging.py
  simple_ml_model.py
  hpcc.py
  student.py
  multiples_while.py
  Assignment3.3.py
  factorial_recursive.py
  hpcc.py
  hpcc.py
  Lab Assignment1.2.py
  lab1
  leap_year.py
  multiples.py
  product_recommender_transparency_fairness...
  review_sentiment_few_shot.py
  review_sentiment_one_shot.py
  review_sentiment_zero_shot.py
  review_sentiment.py
  example1st
  OUTLINE
  TIMELINE
  ZIP EXPLORER

MULTIPLES.PY
1 def print_first_10_multiples() -> None:
2     """Ask the user for a number and print its first 10 multiples using a while loop."""
3     try:
4         n = int(input("Enter a number: ").strip())
5     except ValueError:
6         print("Invalid number entered.")
7         return
8     i = 1
9     while i <= 10:
10        print(n * i)
11        i += 1
12
13
14 if __name__ == "__main__":
15     print_first_10_multiples_while()
16
PROBLEMS OUTPUT TERMINAL
PS C:\Users\saiiku\OneDrive\Desktop\AI> python multiples.py
Enter a number: 5
5
10
15
20
25
30
35
40
45
50
Ln 12, Col 15 Spaces: 4 UTF-8 CRLF Python conda (3.14.2) B4 Go Live 09:41 04-02-2026
```

Task Description #3: Conditional Statements (Age Classification)

Scenario

You are building a basic classification system based on age.

Prompt

1 – if-elif-else

Write a Python program that takes age as user input and classifies the person into Child (0–12), Teenager (13–19), Adult (20–59), or Senior (60+) using nested if-elif-else statements.

2 – Alternative Approach

Rewrite the age classification program using a different approach such as dictionary-based range mapping or simplified logical conditions. Take age as user input.

3 – Explanation

Explain how the conditions in the age classification program work and why the ranges do not overlap.

The top screenshot shows a Python program using nested if-elif-else statements to classify age. The code is as follows:

```
def classify_age():
    print("Invalid input. Please enter an integer age.")
    return

    if age < 0:
        print("Invalid age. Age cannot be negative.")
    else:
        if age <= 12:
            print("Child (0-12)")
        elif age <= 19:
            print("Teenager (13-19)")
        elif age <= 59:
            print("Adult (20-59)")
        else:
            print("Senior (60+)")

if __name__ == "__main__":
    classify_age()
```

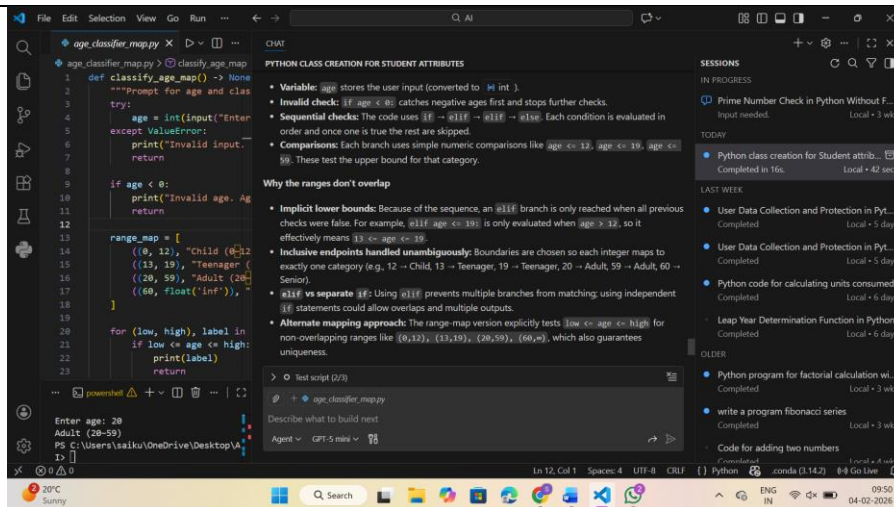
The bottom screenshot shows a Python program using a dictionary-based range mapping approach. The code is as follows:

```
def classify_age_map():
    """Prompt for age and classify using a range->label mapping."""
    try:
        age = int(input("Enter age: ").strip())
    except ValueError:
        print("Invalid input. Please enter an integer age.")
        return

    if age < 0:
        print("Invalid age. Age cannot be negative.")
        return

    range_map = [
        ((0, 12), "Child (0-12)"),
        ((13, 19), "Teenager (13-19)"),
        ((20, 59), "Adult (20-59)"),
        ((60, float('inf')), "Senior (60+)")
    ]

    for (low, high), label in range_map:
        if low <= age <= high:
            print(label)
            return
```



Task Description #4: For and While Loops (Sum of First n Numbers)

Scenario

You need to calculate the sum of the first n natural numbers.

Prompt

1 – For Loop

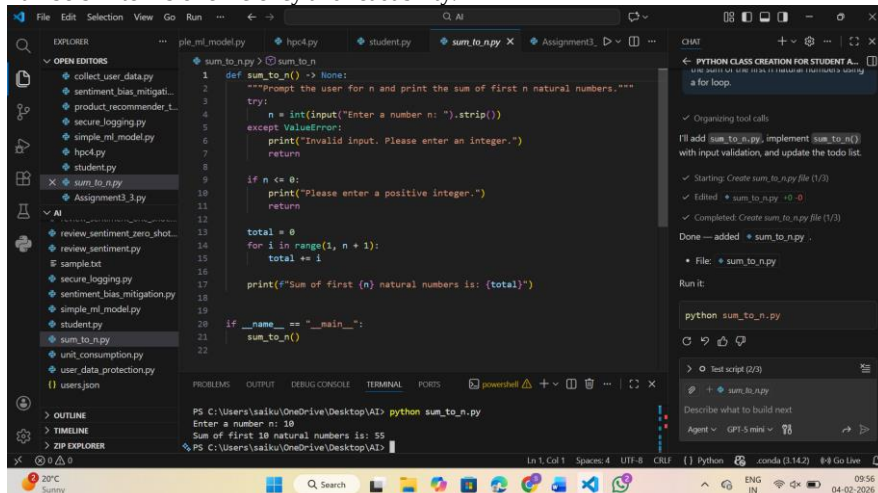
Write a Python function `sum_to_n()` that asks the user to enter a number n and calculates the sum of the first n natural numbers using a for loop.

Prompt 2 – Alternatives

Provide two more versions of the `sum_to_n()` program: one using a while loop and another using the mathematical formula $n(n+1)/2$. Use user input.

Prompt 3 – Comparison

Compare the loop-based and formula-based approaches for calculating the sum of n natural numbers in terms of efficiency and readability.



The image displays three sequential screenshots of a Visual Studio Code (VS Code) editor interface, illustrating the development of a Python program to calculate the sum of the first n natural numbers.

Top Screenshot: The editor shows the file `sum_to_n_formula.py`. The code uses a formula to calculate the sum of the first n natural numbers. The AI chat panel on the right provides instructions and feedback, including a comparison of the formula and loop methods.

```
def sum_to_n_formula() -> None:
    """Prompt the user for n and compute the sum of first n natural numbers using the formula"""
    try:
        n = int(input("Enter a positive integer n: ").strip())
    except ValueError:
        print("Invalid input. Please enter an integer.")
        return
    if n <= 0:
        print("Please enter a positive integer.")
        return
    total = n * (n + 1) // 2
    print(f"Sum of first (n) natural numbers is: (total)")

if __name__ == "__main__":
    sum_to_n_formula()
```

Middle Screenshot: The editor shows the file `sum_to_n_while.py`. The code uses a while loop to calculate the sum of the first n natural numbers. The AI chat panel on the right provides instructions and feedback, including a comparison of the formula and loop methods.

```
def sum_to_n_while() -> None:
    """Prompt the user for n and compute the sum of first n natural numbers using the while loop"""
    try:
        n = int(input("Enter a positive integer n: ").strip())
    except ValueError:
        print("Invalid input. Please enter an integer.")
        return
    if n <= 0:
        print("Please enter a positive integer.")
        return
    total = 0
    i = 1
    while i <= n:
        total += i
        i += 1
    print(f"Sum of first (n) natural numbers is: (total)")

if __name__ == "__main__":
    sum_to_n_while()
```

Bottom Screenshot: The editor shows the file `sum_to_n_while.py`. The code uses a while loop to calculate the sum of the first n natural numbers. The AI chat panel on the right provides instructions and feedback, including a comparison of the formula and loop methods.

```
def sum_to_n_while() -> None:
    """Prompt the user for n and compute the sum of first n natural numbers using the while loop"""
    try:
        n = int(input("Enter a positive integer n: ").strip())
    except ValueError:
        print("Invalid input. Please enter an integer.")
        return
    if n <= 0:
        print("Please enter a positive integer.")
        return
    total = 0
    i = 1
    while i <= n:
        total += i
        i += 1
    print(f"Sum of first (n) natural numbers is: (total)")

if __name__ == "__main__":
    sum_to_n_while()
```

The AI chat panel in the bottom screenshot provides a detailed comparison of the two methods:

- Time Complexity:** Formula: $O(1)$ — constant time. Loop: $O(n)$ — time grows linearly with n .
- Space Complexity:** Both use $O(1)$ extra space (constant auxiliary memory).
- Readability:** Formula: very concise and clearly expresses intent ($n(n+1)/2$). Loop: more verbose; easier for beginners to follow step-by-step.
- Performance / Practicality:** Formula is vastly faster for large n (e.g., $n = 10^8$); loop becomes impractical as n grows.
- Numerical / Overflow Considerations:** In Python both are safe because integers are arbitrary-precision; in fixed-width languages the formula can overflow earlier (product before division) — use wider types or reorder operations carefully. The loop may also overflow but can sometimes be mitigated by incremental checks.
- When to use each:** Use the formula in production or when you only need the final sum. Use a loop when you need intermediate values, are demonstrating algorithmic thinking, or when the sequence is non-standard and has no closed form.
- Recommendation:** Prefer the formula for correctness and efficiency; use the loop for pedagogy or when intermediate results are required.

Task Description #5: Classes (Bank Account Class)

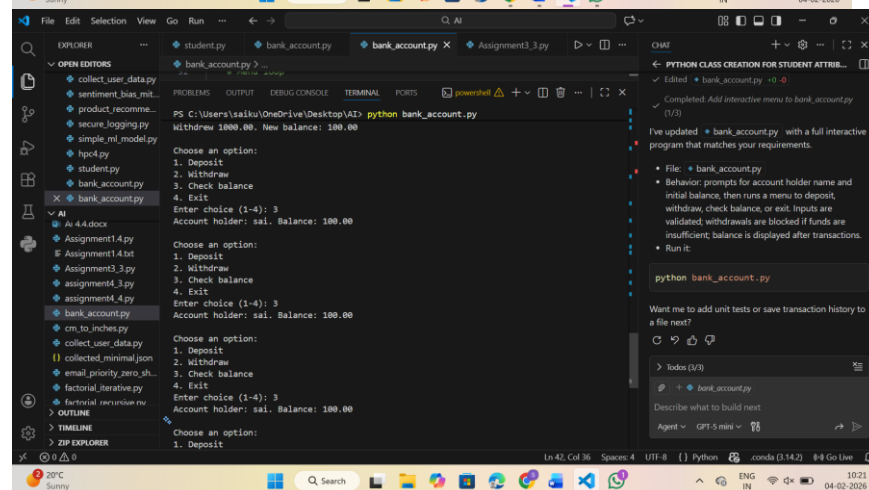
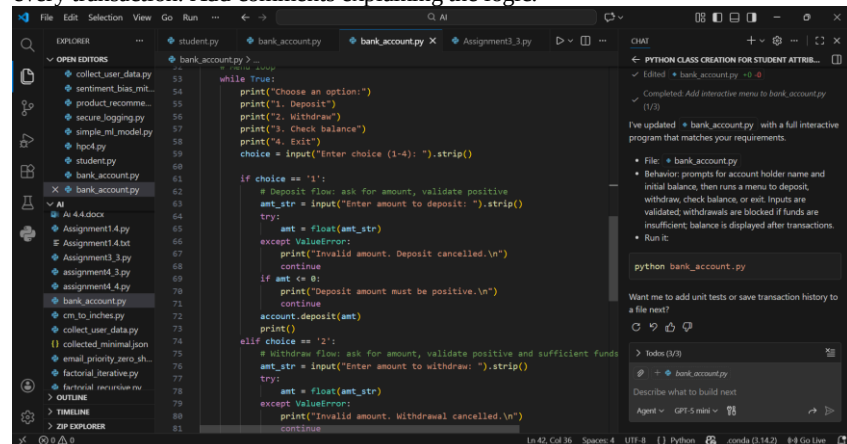
Scenario

You are designing a basic banking application.

Prompt : Write a complete Python program that defines a class BankAccount with attributes account_holder_name and balance. Use a constructor to initialize these values. Include methods deposit(amount), withdraw(amount), and check_balance(), ensuring withdrawal is not allowed if the balance is insufficient.

In the main program, take user input for account holder name and initial balance. Then run a looped menu system where the user can repeatedly choose to deposit money, withdraw money, check balance, or exit.

For deposit and withdrawal operations, ask the user to enter the amount dynamically each time. Validate inputs to prevent negative deposits and overdrawing. Display the updated balance after every transaction. Add comments explaining the logic.



Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.